



Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation
Université Cadi Ayyad
Génie Cyber-Défense et Systèmes de Télécommunications Embarqués



Module Linux et Programmation Avancée

Chapitre 3

Système de gestion de fichiers



Pr. Omar ACHBAROU

o.achbarou@uca.ma

Système de gestion de fichiers

- Un système de gestion de fichiers permet de conserver un ensemble d'informations dans un **fichier** en leur donnant un **nom**.
- L'ensemble des répertoires et fichiers constitue le **système de fichiers** (en anglais File System).
- Le système de gestion de fichiers d'UNIX est caractérisé par:
 - Un mécanisme de sécurité efficace.
 - une interface unique pour toutes les opérations d'entrée-sortie.

Système de gestion de fichiers

3

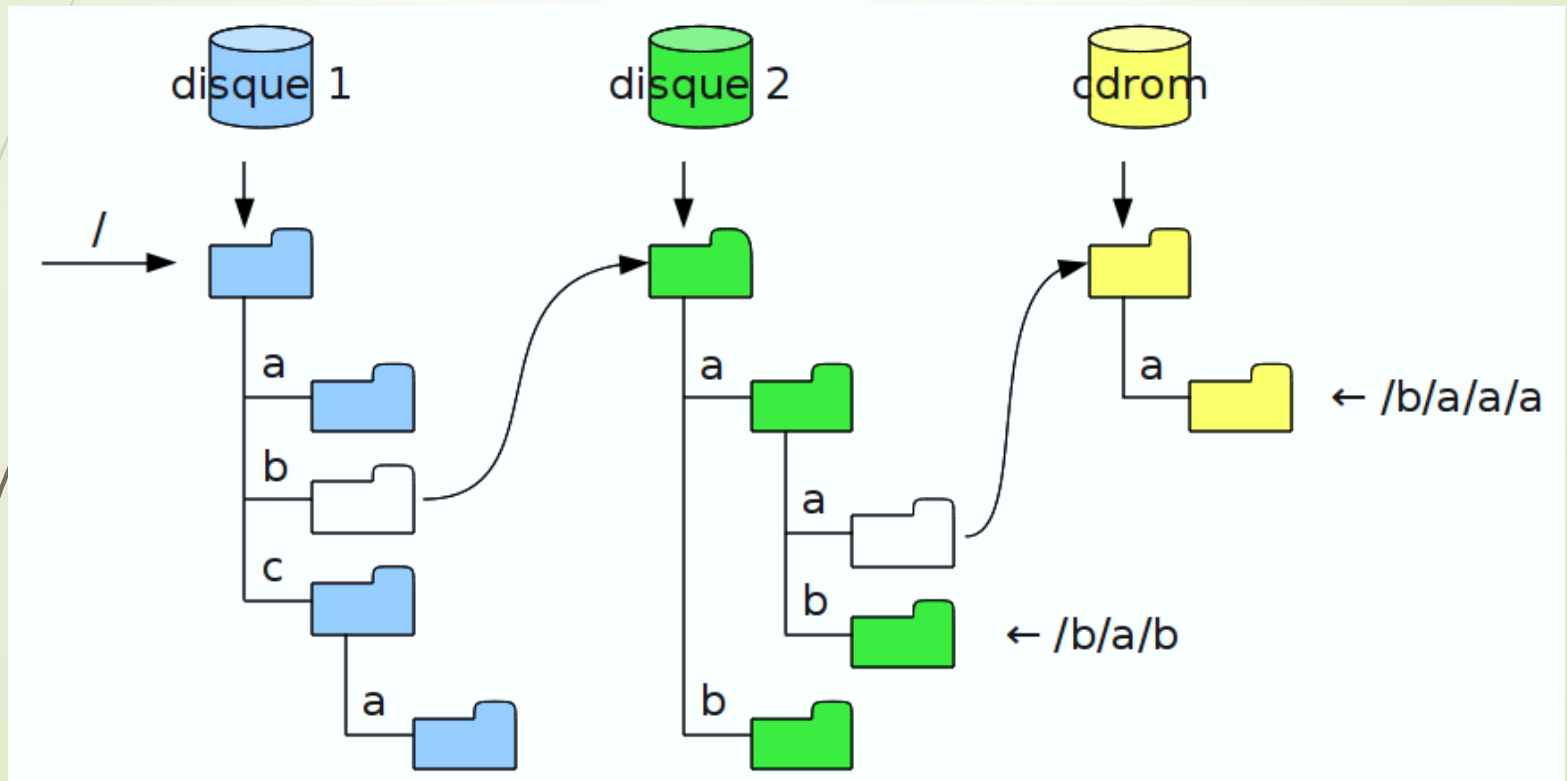
- Il y a 3 types de fichiers :
 - les fichiers ordinaires (**file**).
 - Les répertoires (**directory**).
 - Les **fichiers spéciaux** (device) permettent d'accéder à un périphérique — unité d'entrée-sortie (écran, imprimante, clavier, souris, modem...), unité de stockage (disque dur, clef USB, CD-ROM, DVD-ROM, disquette...).
 - Fichier mode **caractère**
 - Fichier mode **binaire**
 - ...

Structure arborescente sous Unix

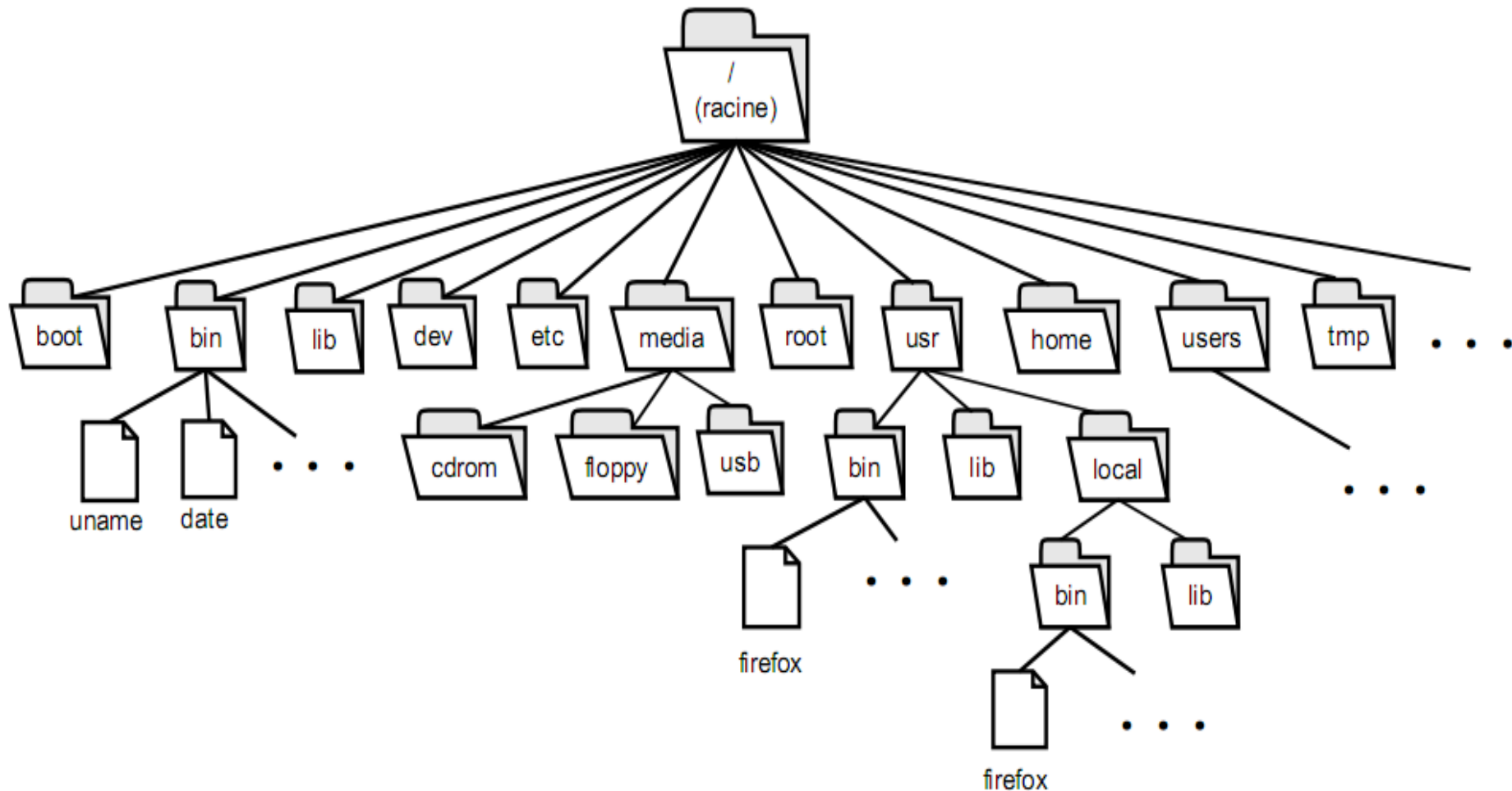
- Le système de gestion de fichiers d'UNIX range les fichiers sous une structure arborescente, à la manière après de **MS-DOS**.
- Les fichiers sont regroupés dans des répertoires.
- Les répertoires peuvent regrouper également d'autres répertoires, dit **sous répertoires**, etc.
- Le système de fichiers est organisé hiérarchiquement sous la forme d'un **arbre**.
- Dans cet arbre la **racine** et les **nœuds** sont des répertoires et les feuilles sont des fichiers ou des répertoires vides.
- Point de départ d'une arborescence.

Hiérarchie sous UNIX

- On a une hiérarchie **unique** ; chaque périphérique ancre sa racine dans un répertoire (**montage**).



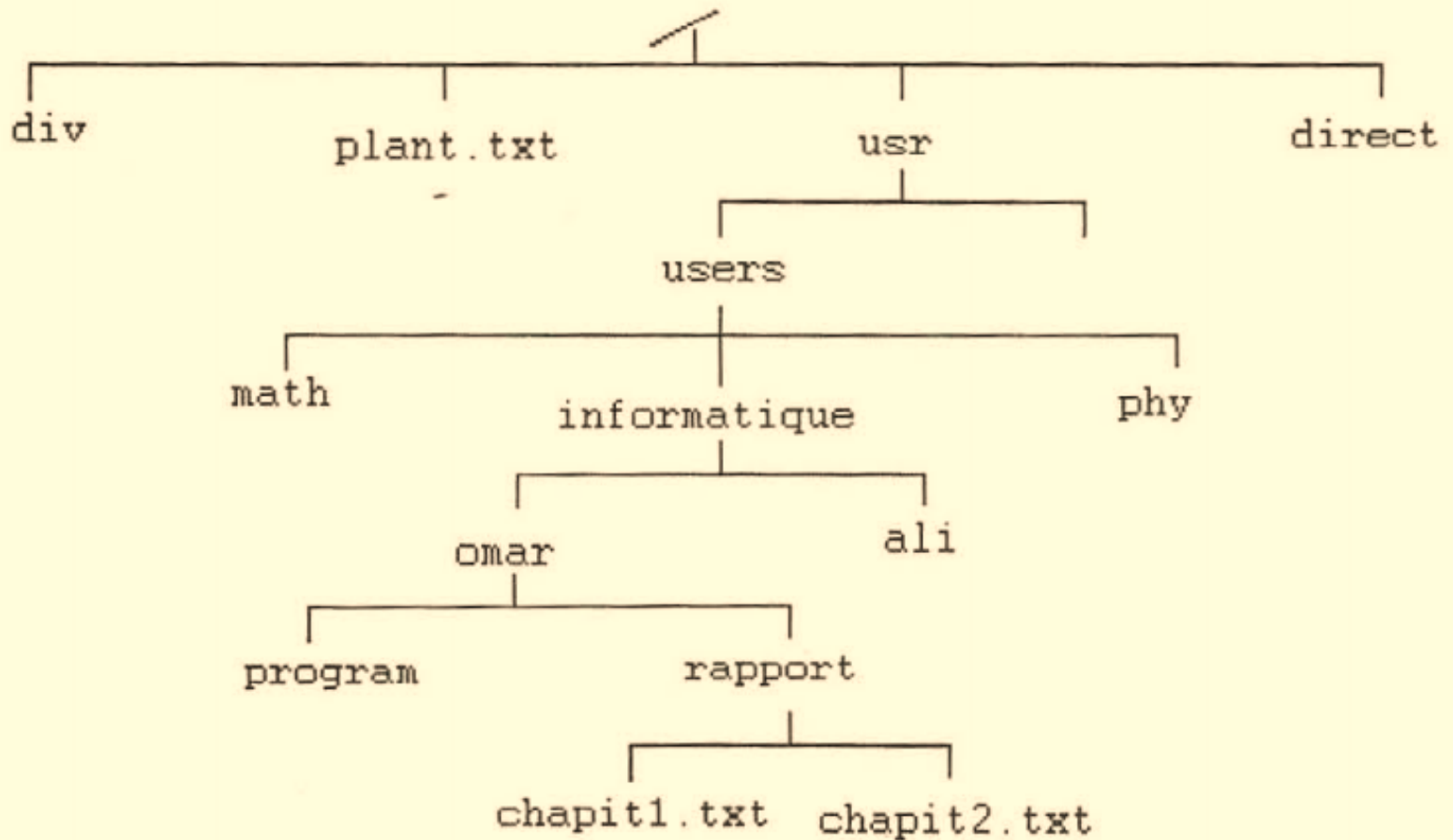
Exemple d'hiérarchie sous UNIX



(source : Univ Grenoble Alpes, IUT2)

Exemple d'hiérarchie sous UNIX

7



➤ Une branche de cet arbre :

`/usr/users/informatique/omar/rapport/chapt1.txt`

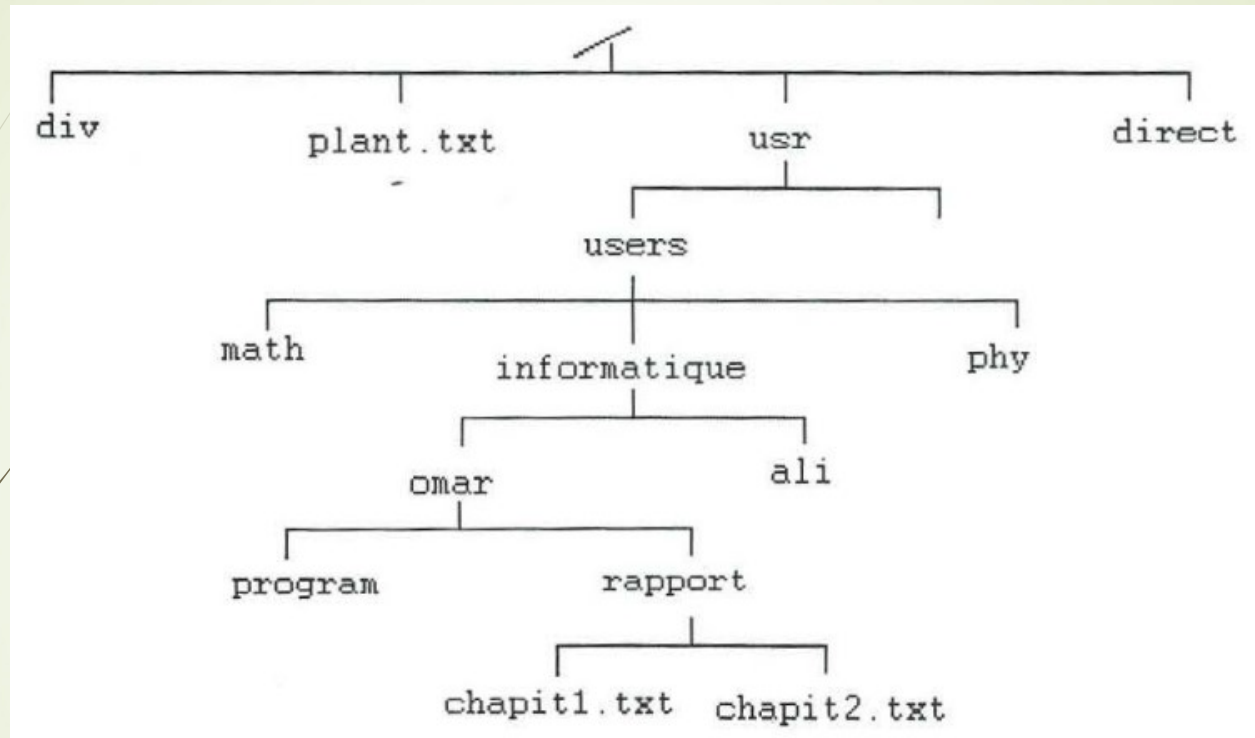
Chemin d'accès

8

- Le **nom complet**, le **chemin d'accès**, d'un fichier ou d'un répertoire est une liste de nom de répertoires séparés par le symbole / et se terminant par le nom du fichier ou du répertoire.
- Un chemin est un seul mot sans espace.
- Il y a 2 types de chemin d'accès :
 - **Chemin d'accès absolu**, commençant par / qui est le répertoire **racine (root)** ;
 - **Chemin d'accès relatif**, par rapport au répertoire courant.

Chemin d'accès

9



Exemple:

- `/usr/users/informatique/omar/rapport/chapit1.txt` → **chemin absolu**
- `rapport/chapit1.txt` → **chemin relatif** si le répertoire courant est `/usr/users/informatique/omar`



Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation
Université Cadi Ayyad
Génie Cyber-Défense et Systèmes de Télécommunications Embarqués



Module Linux et Programmation Avancée

Chapitre 3

Part 1



Pr. Omar ACHBAROU

o.achbarou@uca.ma

Répertoire de base

11

- Chaque utilisateur a son propre répertoire appelé répertoire de base créé par l'administrateur du système lors de l'ouverture de son compte.
- Tous les répertoires et les fichiers d'un utilisateur sont regroupés dans son **répertoire de base** qui porte, en général, le nom de connexion de l'utilisateur.

Ex: /home/omar/

- L'utilisateur peut se déplacer dans l'arborescence avec des restrictions de protection..

FS : Concept d'inode

- Le noyau *Unix* affecte à chaque fichier ou répertoire un **numéro unique** appelé **inode** (index node, nœud d'index) qui permet l'allocation concurrente et l'accès direct (ou aléatoire) à un fichier.

0	Inoccupé
1	Inode
2	
3	Inoccupé
4	Inode
5	Inode
6	Inode
	...

=>

Type de fichier
Droits d'accès
Nombre de liens
UID
GID
Taille
atime
mtime
ctime
Adresse

FS : Concept d'inode

13

La description d'un inode est donnée par :

- le nom du fichier ou du répertoire ;
- le nom du propriétaire ;
- le nom du groupe principal du propriétaire ;
- le numéro d'inode ;
- le jeu des protections ;
- la taille en octet
- le nombre de liens durs ou d'entrées ;
- le fichier ou le répertoire d'origine lors d'un lien symbolique;
- le type: fichier, répertoire, lien symbolique, etc.
- la date de la création ;
- la date de la dernière modification ;
- la date du dernier accès.

Stockage des métadonnées

- Les **métadonnées d'un répertoire** sont stockées dans des **i-nodes** pour les FS NTFS (Windows modernes) ou ext4 (Linux)
- Les métadonnées (y compris le **nom**) de chaque fichier sont stockées directement dans **le contenu du répertoire** le contenant.

contenu du répertoire C:\

Nom	Type	Taille	Création	...
autoexec.bat	F	31 o	12/04/95	...
config.sys	F	49 o	12/04/95	...
donnees	R	2 Ko	26/04/95	...
...				

Métadonnées : I-nodes

15

- Les **méta-données** (sauf le nom) de chaque fichier sont stockées dans un bloc indépendant appelé **i-node**.
- Chaque **i-node** est identifié par un **numéro unique** sur le périphérique (tableau des **i-nodes**).
- Le contenu d'un répertoire ne contiennent qu'une correspondance entre **nom local** du fichier et **numéro d'i-node**

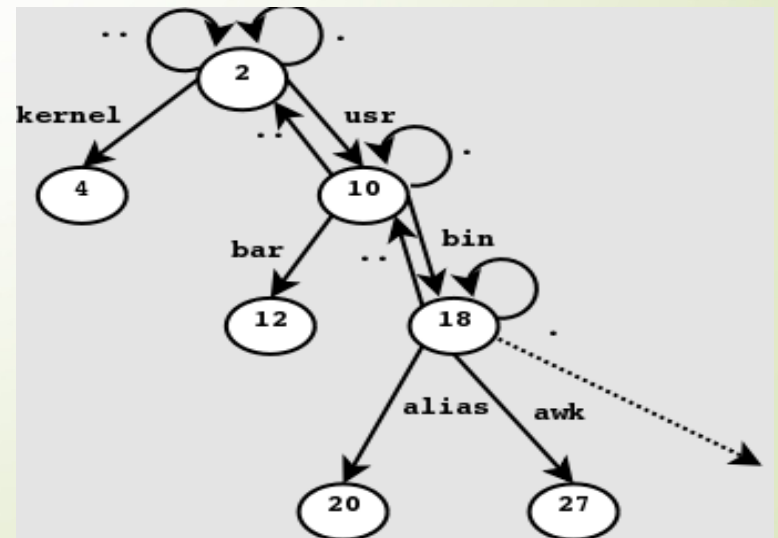
Illustration

contenu du répertoire C:\		i-node 27	i-node 61
nom	i-node	F	F
autoexec.bat	61	49 o	31 o
config.sys	27	12/04/95	12/04/95
donnees	531	...	...
...			
contenu du répertoire C:\donnees		i-node 531	i-node 567
nom	i-node	R	F
cv.doc	567	2 Ko	35 Ko
toto.txt	61	26/04/95	02/01/98
...		...	...

FS : Concept d'inode

16

- **Le nom d'un fichier** est un identificateur. Il y a une différence entre les **lettres en majuscule** et celles en **minuscule**. Tout caractère, y compris les espaces, peut être utilisé dans le nom
- Au point de vue de système, un fichier est une chaîne de caractères non structurés et un répertoire est un **fichier** contenant une suite de couples formés d'un numéro d'inode et d'un nom.
- Suivant l'encodage, un caractère est codifié dans un octet ou plus.





Commandes UNIX

Commande

- Une commande Unix est un mot ou une phrase à la syntaxe bien particulière entrée dans un **interpréteur de commandes** et donnant l'ordre d'actions à exécuter par l'ordinateur
- L'utilisateur peut taper maintenant toutes sortes de commandes qu'il désire. L'interpréteur de commandes **Shell** va les interpréter pour que le système les exécute
- En général, une commande est l'appel de l'exécution du programme correspondant au fichier exécutable

Commande

- Une commande UNIX se décompose en 3 parties :

\$ commande -option1 -option2 -option3 argument1 argument2

- La **commande** elle-même (c'est toujours le premier élément sur la ligne),
- Des **options** commençant par le symbole moins « - »
- Des **arguments** : un nombre fixe ou variable d'arguments.

Commande

➤ Exemple :

➤ ls

➤ ls -l

➤ ls -li rapport

➤ ls -l -i rapport

➤ ls --help

- 
- La commande, les options et les arguments sont séparés par au moins un espace.
 - Il est possible d'accéder aux commandes précédemment tapées grâce à la flèche vers le haut.

Aide sur les commandes

- Pour connaître les commandes qui concernent un **sujet particulier**, vous pouvez utiliser la commande **apropos**.

\$ apropos commande

- Par exemple, si vous souhaitez connaître toutes les commandes qui traitent du calendrier :

\$ apropos calendar

Aide sur les commandes

➡ Manuel des commandes

Pour tout connaître sur une commande, une page de manuel est fournie avec le système.

\$ man commande

➡ Pour tout savoir sur la commande :

\$ man ls

Raccourcis Unix

23

➡ Sous Linux :

Ctrl - Alt - F2: Passe du mode graphique au mode terminal.

Ctrl - Alt - F1: Passe du mode terminal au mode graphique.

Les jokers : Les caractères spéciaux

24

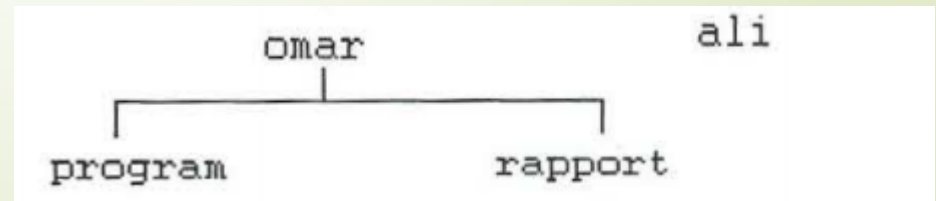
➡ . : le répertoire courant

Le répertoire courant « . » désigne le répertoire dans lequel vous vous trouvez. Il peut être utilisé pour une commande lorsque l'on a besoin de spécifier le répertoire où l'on se trouve.

➡ .. : le répertoire parent d'un répertoire

Le répertoire parent « .. » désigne le répertoire qui se trouve juste au-dessus dans l'arborescence.

Par exemple, le répertoire parent du répertoire **rapport** est le répertoire **omar**



Les jokers : Les caractères génériques

? : un et un seul caractère

***** : une chaîne de caractères

[...] : une liste restreinte de caractères.

Exemples

[a-z]*[0-9] : tous les fichiers qui commencent par une lettre en minuscule et se terminent par un chiffre

. : tous les fichiers avec extension

***** : tous fichiers avec ou sans extension

fichier?.c : liste tous les fichiers dont le nom est "fichier" + un caractère quelconque

fichier.[ch] : liste tous les fichiers commençant par fichier et dont l'extension est "." et est suivie de c ou h
fichier.c fichier.h

Exercice : caractères de substitution (Joker)

1. Lister tous les fichiers commençant par **m**.
2. Lister tous les fichiers ou répertoires de quatre caractères commençant par **d**.
3. Tous les fichiers d'au moins trois caractères et commençant par **e**.
4. Tous les fichiers finissant par **1** ou **2**.
5. Tous les fichiers commençant par les lettres de **a** à **c**, possédant au moins un second caractère avant la terminaison par **1** ou **2**.
6. Tous les fichiers ne finissant pas par **5**.
7. Tous les fichiers commençant par les lettres de **a** à **z** majuscule ou minuscule et finissant par un chiffre.

```
ls m*
```

```
ls d???
```

```
ls e??*
```

```
ls *[12]
```

```
ls [a-c]?*[12]
```

```
ls *[!5]
```

```
ls [Aa-Zz]*[0-9]
```

Les jokers : Les caractères génériques

- **~** : Le répertoire personnel. Le caractère « ~ » (AltGr-2) permet de désigner votre répertoire personnel.
- Les interpréteurs de commande le remplacent par le nom du répertoire de l'utilisateur courant.
- Le répertoire personnel d'un utilisateur particulier : **~login** (ou ~omar) → /home/omar

Gestion de répertoires

pwd (Print Working Directory): afficher le chemin d'un répertoire

```
$ pwd
```

Elle affichera le chemin absolu du répertoire courant

Gestion de répertoires

mkdir : créer un répertoire

- La commande mkdir permet de créer un (ou plusieurs) répertoires. Il suffit de spécifier le nom du (ou des) répertoire(s) que vous souhaitez créer en argument de la commande.

```
$ mkdir [-p] rep1 rep2
```

p : crée les répertoires parents nécessaires

- Exemple :

```
$ mkdir rep1 rep2
```

```
$ mkdir rep1 /tmp/rep2
```

Gestion de répertoires

cd (Change Directory): se déplacer dans l'arborescence de répertoires

change le répertoire courant et qui devient le répertoire **nom_r** s'il existe. Il devient le répertoire de base par défaut.

```
$ cd nom_r
```

Exemple :

```
$ cd ..
```

change le répertoire courant par le père du répertoire courant, c'est à dire à l'étage au-dessus.

```
$ cd /
```

change le répertoire courant par la racine

```
$ cd ~[nom_u]
```

change le répertoire courant par le répertoire de personnel de l'utilisateur nom_u.

Gestion de répertoires

rmdir : supprimer un répertoire

- détruit les répertoires `nom_r_1`, `nom_r_2`, ... s'ils ne contiennent aucun fils, s'ils ne sont pas le répertoire courant ou un ascendant et si l'utilisateur a la permission.

```
$ rmdir nom_r_1 ([nom_r_2] ... ]
```

- détruit les répertoires `nom_r _1`, `nom _r _2`, ... avec leur contenu s'ils ne sont pas le répertoire courant ou un ascendant et si l'utilisateur a la permission.

```
$ rm -r nom_r_l [[nom_r_2] ... ]
```

Gestion de répertoires

32

ls (list) : Visualisation des répertoires et fichiers

```
$ ls [-lrR[Flacdgiu]] [nom_r]
```

liste les fichiers et les sous répertoires du répertoire **nom_r**

-l (long)	affiche en format long (type, date, taille, propriétaire, permissions)
-R (reverse)	affiche les contenus des sous-répertoires
-S (size)	liste les fichiers par taille (les fichiers les plus gros en premier)
-t (temps)	liste les fichiers selon la date de la dernière modification
-U (update)	liste les fichiers selon la date du dernier accès
-a	tous les fichiers même les fichiers cachés (qui commencent par .)
-c	trie de la liste par ordre alphabétique croissant
-ld	description du répertoire courant
-g	donne aussi le groupe
-r (reverse)	Affiche les fichiers en ordre inverse

Gestion de répertoires

33

ls (list) : Visualisation des répertoires et fichiers

- il est possible de contracter les options qui ne font qu'une lettre

\$ ls -ltr

Affiche les noms des fichiers et dossiers en format long, en ordre de modification chronologique inverse.

- Les types de fichiers
 - **-** : fichier ordinaire **d** : répertoire;
 - **l** : lien symbolique ;
 - **C** : fichier spécial structuré avec transfert de caractères pour les entrées-sorties sur disque ;
 - **b** : fichier spécial non structuré avec transfert de blocs pour les entrées-sorties sur terminal ;
 - **s** : fichier socket ;

Gestion de répertoires

34

mv : déplacer ou renommer un fichier et/ou un répertoire

La commande **mv** déplace un fichier existant ou le renomme. Sous sa forme la plus simple, elle a 2 arguments :

- le fichier ou répertoire source.
- le fichier ou répertoire destination

```
$ mv nom_f _ 1 [[nom_f_2] ... ] nom_r
```

- déplace les fichiers nom_f_1, nom_f_2, ... dans le répertoire nom_r.

```
$ mv [-if] nom_f _a nom_f_n
```

- change le nom du fichier nom_f_a qui devient nom_f_n

Gestion de répertoires

35

cp (copy) : copier un fichier et/ou un répertoire

il est possible de contracter les options qui ne font qu'une lettre

```
cp -i -r nom_s nom_d
```

nom_s : fichier ou répertoire source

nom_d : fichier ou répertoire destination

i : mode interactif, avec demande de confirmation

r : copie récursive qui prend le répertoire et tout ce qu'il contient

Gestion de répertoires

36

Remarque :

Comme pour la commande **cp**, **mv** peut avoir **n** arguments. Si **n** est strictement supérieur à **2**, le dernier argument est obligatoirement un répertoire (le répertoire destination)

- ➡ déplace les fichiers `nom_f_1`, `nom_f_2` ... dans `nom_r`

```
$ mv nom_f _ 1 [[nom_f_2] ... ] nom_r
```

TP2 : Exercice 1 – 2 – 3

Gestion de fichiers

38

touch : modifier la date de dernière mise à jour d'un fichier ou le créer

La commande **touch** fixe la date de dernière modification du fichier au moment présent. Si le fichier n'existe pas, la commande crée un fichier vide.

```
$ touch file1.txt
```

Vous pouvez créer ou éditer un fichier texte grâce à la commande **gedit**

```
$ gedit file1.txt
```

Gestion de fichiers

39

rm : supprimer un fichier

Cette commande permet de supprimer un ou plusieurs fichiers, mais aussi des répertoires qui ne sont pas vides

```
$ rm fichier1 fichier2
```

```
$ rm -i
```

Demande confirmation avant de supprimer quoi que ce soit

```
$ rm -f
```

Supprime des éléments sans demander de confirmation

```
$ rm -r
```

supprimer un répertoire et tous les fichiers ou sous-répertoires qu'il contient

```
$ rm -v
```

Affiche des informations au cours du processus de suppression

Gestion de fichiers

40

rm : supprimer un fichier

```
$ rm -rf
```

- Cette commande est peut-être la plus dangereuse des commandes Linux.
- en particulier si vous êtes connecté en tant que « **super utilisateur** » (**root**). Les fichiers supprimés ne pourront pas être restaurés .
- Si vous tapez la commande **rm -rf *** à la racine et que vous êtes super-utilisateur, vous effacez toute l'arborescence (répertoires et fichiers), sans possibilité de retour en arrière.

Gestion de fichiers

41

more : liste le contenu du fichier

- ▀ liste le contenu du fichier nom_f page par page, c'est à dire 23 lignes ou écran par écran.

```
$ more [+num +chaîne] [nom_f]
```

+ num : commence à partir de la ligne numéro

+ /chaîne : commence deux lignes avant la première ligne qui contient au moins une occurrence de chaîne.

q ou **Q** : quitte more

Gestion de fichiers

42

less : liste le contenu du fichier

- liste le contenu du fichier `nom_f` page par page, c'est à dire 23 lignes ou écran par écran.

```
$ less [nom_f]
```

cat : liste le contenu du fichier

- affiche le contenu des fichiers `nom_f_1`, `nom_f_2`, ... d'un seul coup et d'une façon séquentielle.

```
$ cat [-bn] nom_f_1 [(nom_f_2) ... ]
```

b : ignore les lignes blanches, en les affichant sans les numérotées, et numérote les lignes, à partir du début du fichier;

n : numérote les lignes au début.

Gestion de fichiers

43

Remarque :

more, less et cat sont utilisées uniquement pour les **fichiers textes**.

Gestion de fichiers

44

Commande tee

transmet les sorties dans le fichier nom_f jusqu'à Ctrl d, en plus de la sortie standard. Si le fichier existe déjà il sera détruit.

```
tee [-a] nom_f
```

a : ajoute à la fin du fichier. Si le fichier n'existe pas il sera créé.

Exemple :

```
$ tee f.txt  
bien  
bien  
Ctrl d
```

crée un fichier f.txt contenant bien, une seule fois.

```
$ tee -a f.txt  
bien  
bien  
Ctrl d
```

crée un fichier f.txt contenant bien, une seule fois.

Gestion de fichiers

vi : liste le contenu du fichier : est un éditeur de texte plein écran écrit par Bill Joy en 1976 sur une des premières versions de la distribution Unix BSD

Insertion

Activer le mode insertion → **i**

- Insérer au début de la ligne → **I** (i majuscule)
- Activer le mode insertion, un caractère après le curseur → **a**
- Insérer en fin de ligne → **A**
- Insérer une ligne (en dessous du curseur) → **o**
- Insérer une ligne (au dessus du curseur) → **O**
- Quitte le mode édition → **Echap**

Effacer

Effacer le caractère courant → **x**

Effacer la ligne courante → **dd**

Effacer n lignes à partir de la ligne courante → **ndd**

Quitter / Enregistrer

Enregistrer SANS quitter → **:w**

- Quitter en enregistrant les modifications → **:wq** ou **:x** ou **ZZ**
- **Quitter sans enregistrer les modifications** → **:q!**

Rechercher

Recherche en avant du mot texte
→ **/texte**

• Recherche en arrière du mot texte
→ **?texte**

• Aller à l'occurrence suivante → **n**

• Aller à l'occurrence précédente
→ **N**

Gestion de fichiers

46

Commande head

liste les **num** premières lignes des fichiers nom_f_1 , nom_f_2, ... Elle affiche les 10 premières lignes par défaut

```
head [-n] num [nom_f_1] [nom_f_2]
```

Commande tail

liste les **num** dernières lignes des fichiers nom_f_1 , nom_f_2, ... Elle affiche les 10 dernières lignes par défaut

```
tail [-n] num [nom_f_1] [nom_f_2]
```

Gestion de fichiers

47

wc (Word Count)

donne le nombre de lignes, de mots et de caractères, ou d'octets, suivant l'ordre des options, qui se trouvent dans un fichier.

```
wc [-lwc] nom_f_1 nom_f_2...
```

-l donne le nombre de ligne du fichier

Exemple :

```
wc file1.txt
```

```
25 123 1265 file1.txt
```

```
wc file1.txt file2.txt
```

```
25 123 1265 file1.txt
15 13 128 file2.txt
40 136 1393 total
```

Lien symbolique

48

- Un lien est un fichier spécial contenant comme information un chemin vers un autre fichier.
- C'est une sortie d'alias. Il existe deux types de lien
 - le lien **dur** (hard Link)

In fichier lien

- le lien **symbolique** (soft link).

In -s fichier lien

TP2 : Exercice 4 – 5 – 6



Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation
Université Caddi Ayyad
Génie Cyber-Défense et Systèmes de. Télécommunications Embarqués



Module Linux et Programmation Avancée

Chapitre 3

Système de gestion de fichiers



Pr. Omar ACHBAROU

o.achbarou@uca.ma